Pipes in the Desert
Requirements, Project, Functionality

TEAM A

Supervisor:
Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

February 27th, 2026

2. Requirements, project, functionality

2.1 Introduction

2.1.1 Goal

The goal of this document is to describe the requirements, scope, functionality, and constraints of the “Pipes in the Desert” software system. This document defines what the system must do, who will use it, and what conditions must be satisfied during development and execution. It provides a clear basis for the design, implementation, and testing of the system.

2.1.2 Application domain

The application is a strategy-based simulation game that models a desert water distribution system. The system simulates water transfer from mountain springs to city cisterns through a dynamically modifiable pipe network. Two opposing teams interact within this environment: one team attempts to maximize water delivery to cisterns, while the other team attempts to sabotage the system and increase water loss.

The software can be used for:

- Educational purposes (modeling systems and resource flow)
- Demonstration of object-oriented design
- Simulation-based project assignments
- Strategy and decision-based gameplay

The primary purpose is to simulate system behavior under dynamic structural changes and conflicting user interactions.

2.1.3 References

[1] Budapest University of Technology and Economics, “Problem definition.” [Online]. Available: https://www.iit.bme.hu/targyak/BMEVIIIAB02/problem-definition

[2] PlantUML, “PlantUML.” [Online]. Available: https://plantuml.com/

[3] I. Sommerville, Software Engineering, 8th ed. Pearson Education Ltd., 2007.

[4] Object Management Group, “UML 2.1.1 Superstructure Specification & Infrastructure Specification,” 2007. [Online]. Available: http://www.omg.org/technology/documents/modeling_spec_catalog.htm#UML

[5] LLM was used for proofreading.

2.1.4 Document content

This document contains:

- An overview of the system and its purpose.
- Description of users and system functions.
- Functional and non-functional requirements.
- Essential use-cases and their descriptions.

- Glossary of important domain terms.
- Project planning and protocol information.

2.2 Overview

2.2.1 General overview

The game application is structured into multiple subsystems:

- A Graphical User Interface (GUI) for visual interaction.
- A User Interface (UI) for handling user input.
- A Pipe Network (Grid) that includes:
    - Pipes
    - Water sources
    - Cisterns (water storage units)
    - Pumps (flow control elements)

A Player subsystem managing player roles and actions.

Different behaviors are categorized conceptually, such as:

- Water transfer behavior,
- Player movement behavior,
- System modification behavior.

The GUI serves as the primary interaction medium, allowing players to control the game using mouse clicks, keyboard input, and graphical elements.

The system stores runtime data such as:

- Water quantities (scores),
- Player names and team information.

2.2.2 Functions

This game simulates a conflict between two groups attempting to influence the efficiency of a water delivery network.

Water originates from source nodes and moves through transport channels toward storage facilities. Along the path, flow regulators (pumps) determine the direction and transfer of water.

Key characteristics of the system:

- Transport channels have limited capacity.
- Flow regulators can connect to multiple channels, but actively route water between only one selected incoming and one outgoing connection.
- Regulators may temporarily malfunction, blocking transfer.
- Each regulator contains an internal buffer for intermediate storage.
- Water transfer is limited by downstream capacity constraints.

The network structure is not static. Connections may be reconfigured, extended, or disrupted during gameplay.

Expansion Mechanics

- Storage facilities periodically generate new infrastructure components.
- Transport channels can be reattached to different elements.
- Flow regulators may be inserted into existing channels by splitting them.

Player Roles

Two opposing groups participate:

Maintenance Team

Responsible for maximizing water delivery. They may:

- Repair malfunctioning regulators
- Seal damaged transport channels
- Reconfigure flow directions
- Extend the infrastructure
- Install new regulators

Disruption Team

Focused on reducing system efficiency. They may:

- Damage transport channels
- Alter flow routing
- Interfere with the regulator configuration

Movement Rules

- Players may only move along existing infrastructure.
- Transport channels allow only one player at a time.
- Regulatory nodes can host multiple players simultaneously.
- Movement is turn-based.

Game Outcome

The game runs for a predefined duration.
The winning team is determined by comparing:

- Total water successfully stored
- Total water lost from the system

2.2.3 Users

The system has the following user types:

Plumbers

- can set the directions of the pumps.
- can fix the leaking pipes.
- can change the pipes.
- can pick up new pumps.

Saboteurs

- can change the directions of the pumps.
- can puncture the pipes.

2.3 Requirements

2.3.1. Functional requirements

ID	Description	Check	Priority	Source	Use-case	Comment
FR-01.1	The system shall allow players to enter their names, choose a role (plumber or saboteur), set game duration, and start the game.	The game starts only when the duration is set	Should have	Group Meeting	Start/ Setup Game	
FR-01.2	Each team must have at least 2 players.	The game starts only when both teams have 2+ players.	Must Have	Problem Definition	Start/ Setup Game	
FR-02	The system shall generate a starting pipe network with at least one spring, one cistern, one pump, and connecting pipes. .	Network is visible and complete at game start	Must have	Problem Definition	Start/ Setup Game	

FR-03	Water shall flow from springs through pipes and pumps toward cisterns each turn. Water reaching a cistern is counted as delivered; water reaching a free pipe end is lost to the desert.	Cistern and lost water counters update each turn correctly	Must have	Problem Definition	Start/ Setup Game	
FR-04	A pump shall have one active input and one active output pipe. Water transfers only between these two. A pump shall stop transferring water if it is broken or if the output pipe is full.	No water passes through a broken pump or a full output; only active pipes carry water	Must have	Problem Definition	Start/ Setup Game	
FR-05	The system shall randomly break pumps during the game. A broken pump does not transfer water until repaired.	Pump stops working after a random malfunction event	Must have	Problem Definition	Start/ Setup Game	

FR-06	A plumber shall be able to repair a broken pump or fix a leaking pipe when standing on it.	Pump resumes transfer after repair; pipe stops leaking after fix	Must have	Problem Definition	Repair Pipe/Pump	
FR-07	A plumber shall be able to change a pump's active input or output pipe when standing on that pump.	Water direction changes after plumber redirects pumps	Must have	Problem Definition	Change Pump/Pipe Direction	
FR-08	A plumber shall be able to connect one of its ends to a network element.	New pipe appears in network after plumber places it	Must have	Problem Definition	Place New Pipe	
FR-09	A plumber shall be able to insert a new pump into an existing pipe by splitting it in two and connecting both ends to the pump.	New pump visible in network; original pipe replaced by two segments	Must have	Problem Definition	Insert New Pump	
FR-10	A saboteur shall be able to puncture a pipe when standing on it. A punctured pipe leaks all the water to the desert.	All water is lost in the punctured pipe; none reaches the far end	Must have	Problem Definition	Puncture a Pipe	

FR-11	A saboteur shall be able to change a pump's active input or output pipe when standing on that pump.	Water direction changes after the saboteur redirects the pump	Must have	Problem Definition	Change Pump/Pipe Direction	
FR-12	Players may only move along pipes and pumps. Only one player may occupy a pipe at a time; multiple players may share a pump. The game is turn-based with one action per player per turn.	Player blocked from occupied pipe; multiple players allowed on pump; one action per turn enforced	Must have	Problem Definition	Move Player	
FR-13	Cisterns shall periodically produce new pipes and pumps that plumbers can pick up and use.	New pipe/pump appears at cistern after a set interval	Must have	Problem Definition	Generate New Pipes/Pumps	
FR-14	The system shall track and display in real time the total water delivered to cisterns and total water lost to the desert.	Both counters update each turn correctly	Must have	Problem Definition	Declare Winning Team / End Game	
FR-15	When the game timer expires, the	Correct winner	Must have	Problem Definition	End Game/ Declare Winner	

	system shall compare delivered water vs. lost water and declare the winning team.	shown at game end				
FR-16	The system shall display the full pipe network, all player positions, scores, and remaining time on screen. Players shall perform all actions through the GUI using mouse or keyboard.	All elements visible and updated in real time; all actions accessible via GUI	Must have	Problem Definition	Start/Supervise Game	
FR-17	System must enforce each pump's maximum number of connectable pipes, connecting beyond limit should be prevented.	If attempted to exceed limit: system refuses connection	Must have	Problem Definition	Start/Supervise Game	

2.3.2 Resource requirements

ID	Description	Check	Priority	Source	Comment
RR-00	The game must compile and run on the virtual machine found on the smallville cloud	This can be checked by creating a virtual machine and compiling and running the	Must have	Course webpage	

	(https://smallville.cloud.bme.hu/). The machines have Windows 10 with Java 11.	program in the Windows 10 with Java 11 virtual machine.			
RR-01	The system shall run on a machine with at least an Intel Core i3 at 1.5 GHz, 2 GB RAM, and 200 MB free disk space.	Application launches and runs without errors on minimum hardware	Should have		4 GB RAM recommended
RR-02	The system shall support a minimum screen resolution of 1024×768 and be fully operable with a standard keyboard and mouse.	GUI renders correctly; all actions are performable with keyboard and mouse	Should have	Group Meeting	
RR-03	The system shall use WinForms or WPF as the GUI framework and shall be developed using Visual Studio 2022 or JetBrains Rider.	GUI window opens correctly; project builds in Visual Studio 2022	Could have	Group Meeting	

RR-04	The system shall be compilable via 'dotnet build' or Visual Studio, and runnable via 'dotnet run', Visual Studio (F5), or directly as an .exe.	All three run methods successfully launch the application	Should have	Group Meeting	
RR-05	The system shall operate fully offline on a single machine. All players share one keyboard and mouse during gameplay.	Application runs with no network; 4 players can take turns on one machine	Should have	Group Meeting	

2.3.3. Non- functional requirements, Restrictions

ID	Description	Check	Priority	Source	Comment
NF-01	A new player shall be able to understand the basic rules and start playing without external help within 10 minutes	New user plays a full round without needing instructions after 10 min familiarizatio n	Must have	Group Meeting	
NF-02	The system shall not crash or produce unhandled errors during normal gameplay. Any	No unhandled exceptions occur during a full game session	Must have	Group Meeting	

	unexpected error shall display a user-friendly message.				
NF-03	The game shall respond to player input within 1 second. The GUI shall update without noticeable delay after each action.	All actions reflect on screen within 1 second on the minimum hardware	Should have	Group Meeting	
NF-04	The source code shall follow a consistent coding style and be structured in logical modules so that individual components can be tested independently.	Unit tests can be written for core game logic without modifying production code	Must have	Group Meeting	
NF-05	The system shall run on any operating system that supports Java JDK 17 without requiring changes to the source code.	Application compiles and runs on Windows and at least one other OS with Java JDK 17	Must have	Group Meeting	
NF-06	The game shall support a minimum of 2 players per team with no enforced	Game runs with 2 players per team; no in-game chat or network	Must have	Group Meeting	

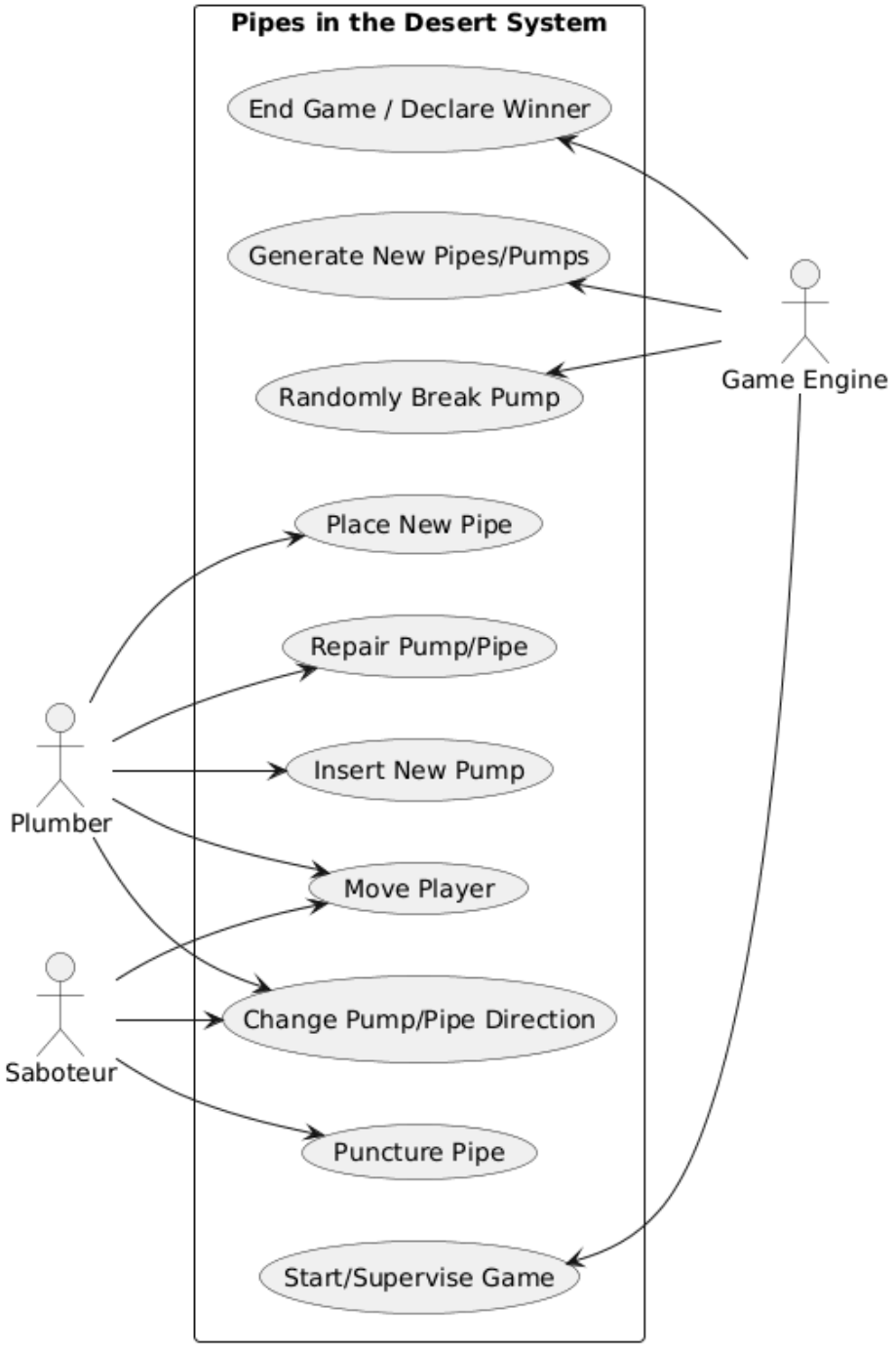
	upper limit. All players share one machine and cannot communicate through the system.	communication exists			

2.4 Essential use-cases

2.4.1. Actors

Actors	**Description**
Plumber	A human player on the maintenance team. Initiates construction and repair actions with the goal of maximising water delivery to cisterns.
Saboteur	A human player on the disruption team. Initiates damage and routing actions with the goal of maximising water loss from the network.
Game Engine	The game engine itself. Autonomously triggers time-based events (pump failures, component generation, game end) in support of the game flow. Reacts to internal timers and state conditions rather than pursuing an independent goal. Initiates and supervises the game session. Has a clear goal: get the game started, keep it running, and determine the outcome. Directly triggers UC01 and can manually end the session.

2.4.2. Use-case diagram



2.4.3 Use-case descriptions

Start / Supervise Game

Use-case name	Start / Supervise Game

Short description	The Game Controller initialises a new game session, configures the pipe network, assigns player roles, and supervises the overall game until the system declares a winner.
Actors	Game Engine
Preconditions	No game session is currently running. Player names and team assignments are known.
Dialog / Scenario	1. Game Controller selects "New Game" and enters player names and team assignments. 2. System generates the initial pipe network: springs, cisterns, pumps, and connecting pipes. 3. System assigns the Plumber role to team 1 and the Saboteur role to team 2. 4. System starts the countdown timer and activates water flow from springs. 5. Game Controller monitors the game state in real time. 6. When the timer expires, the " Declare Winning Team / End Game is triggered automatically by the System.

Move Player

Use-case name	Move Player
Short description	A player moves their character one step along the existing pipe network infrastructure to an adjacent pipe segment or pump node. Movement is turn-based. A pipe segment may only be occupied by one player at a time; pump nodes may host multiple players simultaneously.
Actors	Plumber , Saboteur
Preconditions	It is the player's turn. The destination element exists and is adjacent to the player's current position in the network.
Dialog / Scenario	1. Player selects an adjacent infrastructure element (pipe segment or pump node) as their destination. 2. System validates the move: the element must be connected to the current position, and if it is a pipe segment, it must not be occupied by another player. 3. If valid: System moves the player to the selected element and updates the display. 4. If invalid: System notifies the player; the player may select a different destination or skip movement. 5. Turn passes to the next player.

Place New Pipe

Use-case name	Place New Pipe

Short description	A Plumber collects a new pipe component generated by a cistern and connects it to the existing network to extend water delivery infrastructure.
Actors	Plumber
Preconditions	A new pipe component is available at a cistern. The Plumber is located at that cistern.
Dialog / Scenario	1. Plumber selects "Pick Up Pipe" at the cistern. 2. Plumber moves to the desired attachment location on the network. 3. Plumber selects "Place Pipe" and puts the pipe in the . 4. System validates the placement (both endpoints must connect to existing network nodes). 5. If valid: System adds the pipe segment to the network and recalculates water flow. 6. If invalid: System displays an error; the Plumber retains the component.

Repair Pipe/Pump

Use-case name	Repair Pipe/Pump
Short description	A Plumber repairs a malfunctioning pump/ or a damaged pipe segment, restoring normal water flow through that element. The element may have been damaged by a Saboteur or broken by the system.
Actors	Plumber
Preconditions	The Plumber is located on the target pump or pipe segment. The target element is in a damaged or malfunctioning state.
Dialog / Scenario	1. The plumber selects the "Repair" action on the damaged element they are currently standing on. 2. System validates that the element is damaged and the actor is a Plumber. 3. System restores the element to its working state. 4. System recalculates water flow and removes any water-loss or blockage effect at that element. 5. System updates the visual indicator (element returns to its normal appearance).

Insert New Pump

Use-case name	Insert New Pump

Short description	A Plumber collects a new pump component generated by a cistern and inserts it into an existing pipe segment, splitting the pipe and adding a new flow-control node at that point.
Actors	Plumber
Preconditions	A new pump component is available at a cistern. The Plumber is located at that cistern.
Dialog / Scenario	1. The plumber selects "Pick Up Pump" at the cistern. 2. Plumber moves to the target pipe segment. 3. Plumber selects "Insert Pump"; system splits the pipe at that point and places the pump between the two resulting segments. 4. System assigns a default inlet/outlet configuration to the new pump. 5. System recalculates water flow and updates the display with the new pump node.

Change Pump/Pipe Direction

Use-case name	Change Pump Direction
Short description	A player standing on a working pump reconfigures its active inlet–outlet pair, altering the direction of water flow through that node. Plumbers use this to optimise delivery; Saboteurs use it to divert water away from cisterns.
Actors	Plumber, Saboteur
Preconditions	The player is located on the target pump node. The pump is in a working (non-malfunctioning) state.
Dialog / Scenario	1. Player selects the "Change Pump/Pipe Direction" action on the pump they are standing on. 2. System presents all available inlet–outlet routing options for that pump or the pipe. 3. Player selects the desired configuration. 4. System updates the pump's active routing. 5. System recalculates water flow across the entire network and updates the display.

Puncture Pipe

Use-case name	Puncture Pipe
Short description	A Saboteur damages a pipe segment, causing water to leak from the network at that point and increasing total water loss. The pipe remains damaged until a Plumber performs Repair Pump/Pipe.
Actors	Saboteur
Preconditions	The Saboteur is located on the target pipe segment. The pipe segment is currently intact.
Dialog / Scenario	1. Saboteur selects the "Puncture Pipe" action on the pipe segment they are standing on. 2. System confirms the pipe is intact and the actor is a Saboteur. 3. System marks the pipe as damaged and begins accumulating water loss at that segment each turn. 4. System updates the visual indicator on the pipe (e.g. red highlight). 5. The pipe remains damaged until a Plumber repairs it .

Randomly Break Pump

Use-case name	Randomly Break Pump
Short description	The system autonomously malfunctions a working pump at a random interval, independent of any player action. This introduces unpredictable disruption and creates repair opportunities for the Plumber team.
Actors	Game Engine
Preconditions	At least one pump in the network is in a working state. The game session is active.
Dialog / Scenario	1. System timer triggers a random pump-breakdown event at a predefined interval. 2. System selects a working pump at random from the active network. 3. System sets the selected pump's state to "malfunctioning", halting water routing through it. 4. System updates the visual indicator on the pump (e.g. warning icon or colour change). 5. The pump remains malfunctioning until a Plumber performs Repair Pump/Pipe.

Generate New Pipes / Pumps

Use-case name	Generate New Pipes / Pumps
Short description	The system periodically creates new pipe and pump components at cistern locations, making them available for Plumbers to pick up and deploy. This is the mechanism by which the Plumber team can expand the network.
Actors	Game Engine
Preconditions	The game session is active. A cistern node exists in the network.
Dialog / Scenario	1. System timer triggers a component-generation event at a predefined interval. 2. System determines the type of component to generate (pipe or pump) according to game rules. 3. System places the new component at a cistern node and marks it as available for pickup. 4. System updates the display to indicate the new component at the cistern. 5. Component remains at the cistern until collected by a Plumber.

End Game / Declare Winner

Use-case name	End Game / Declare Winner
Short description	The system terminates the game session when the countdown timer expires, calculates the final scores (total water stored vs. total water lost), and announces the winning team.
Actors	Game Engine
Preconditions	A game session is active. The countdown timer has reached zero, or the Game Controller has manually ended the session.
Dialog / Scenario	1. System detects that the game timer has expired. 2. System halts all water flow, player movement, and automated events. 3. System calculates final scores: total water volume stored in cisterns vs. total water volume lost from the network. 4. System compares scores: if stored water > lost water, the Plumber team wins; otherwise the Saboteur team wins. 5. System displays the results screen showing both scores, the winning team, and a game summary. 6. Game Controller (or players) may choose to start a new game or exit.

2.5 Glossary

A

Action (Turn action) – One operation a Player may perform during their turn (e.g., move, repair, puncture, change direction).
Active Elements – Elements of the Pipe Network that affect water flow or gameplay, such as **Spring**, **Cistern**, and **Pump**.
Active Input Pipe – The pipe currently selected by a Pump as the pipe from which it receives water.
Active Output Pipe – The pipe currently selected by a Pump as the pipe to which it sends water.

B

Broken / Malfunctioning Pump – The state of a Pump when it cannot transfer water until repaired by a Plumber.
Buffer (Water Tank) – The internal temporary storage inside a Pump used during water transfer.

C

Capacity – The maximum amount of water that a Pipe or Pump can transfer during a single simulation step.
Cistern – An active element that stores delivered water for the city and periodically produces new Pipes and Pumps.
Closed Pipes – Pipes connected to a Pump but not selected as the Pump's active input/output, therefore not transferring water through that Pump.
Connectable Pipes – Pipes that can be connected to a Pump; the number is limited by the Pump's connection limit.
Connection Limit – The maximum number of pipes that may be connected to a Pump.

D

Dangerous Environment – The desert conditions that restrict Player movement to the Pipe Network (players cannot move freely outside pipes/pumps).
Damaged / Punctured Pipe – A Pipe that leaks water to the desert until repaired by a Plumber.
Desert (Water Loss) – The area outside the Pipe Network where leaked water is counted as lost.

F

Flow Direction – The selected routing of a Pump that determines which input is connected to which output.
Flowable – An interface/concept representing elements that participate in water transfer in the Pipe Network.

G

Game Engine – The system component that advances turns, calculates water flow, triggers random pump failures, generates components, and ends the game.

Game Timer – The countdown mechanism that determines the duration of a game session and triggers the end-game evaluation when it reaches zero.
Grid (Pipe Network) – The region of the game containing all Pipes, Pumps, Springs, and Cisterns and their connections.
Graphical User Interface (GUI) – The visual part of the system that displays the game state and allows players to interact with the game using graphical elements, the mouse, and keyboard.

I

Input Pipe – A Pipe through which water can enter a Pump.

L

Leaking Pipe – A damaged Pipe that causes water to leave the system and be counted as water loss.

M

Maintenance Team – The team of Plumbers whose objective is to maximize water delivered to Cisterns.
Manufactured Pipes / Pumps – New Pipe or Pump components generated by Cisterns and made available for Plumbers to pick up and deploy.
Movable – An interface / concept representing elements or actors that can change position along the Pipe Network.
Modifiable – An interface / concept representing elements that can be changed by players (e.g., Pump direction, Pipe connections, damage state).

N

Network Reconfiguration – Changing the structure of the Pipe Network by placing new Pipes, reconnecting endpoints, or inserting Pumps.
Non-Functional Requirement – A requirement describing system quality or constraints (performance, usability, reliability, portability).

O

Out of Order – Another name for a malfunctioning Pump state where water cannot pass through the Pump.
Output Pipe – A Pipe through which water exits a Pump.

P

Pipe – A transport element that carries water between Springs, Pumps, and Cisterns.
Pipe System – The full connected water network used in the game, consisting of Pipes and active elements.
Player – A user who controls a role (Plumber or Saboteur) and performs actions in turns.
Plumber – A Player on the Maintenance Team who can repair Pumps, fix Pipes, place new Pipes, insert Pumps, and redirect Pump flow.
Pump – An active element that routes water from one selected input pipe to one selected output pipe, using an internal Buffer and possibly malfunctioning.
Puncture – The sabotage action that damages a Pipe and causes leakage.

R

Repair – The maintenance action that restores a broken Pump or leaking Pipe to a working state.

S

Saboteur – A Player on the Disruption Team who can puncture pipes and change Pump routing to increase water loss.
Spring – A water source element that produces water that flows into the Pipe Network.

T

Team – A group of Players sharing the same goal: Maintenance Team (Plumbers) or Disruption Team (Saboteurs).
Turn-Based Gameplay – The rule that Players act one after another, typically with one action per Player per turn.

U

User Interface (UI) – The part of the system through which players interact with the game. It handles user input (e.g., mouse clicks and keyboard actions) and provides feedback by displaying the current game state, including the pipe network, player positions, scores, and remaining time.

W

Water Loss – Water that leaves the Pipe Network (through leaks or free ends) and is counted against the Maintenance Team.
Winning Criteria – The rule that determines the winner at the end: comparison of total water stored in Cisterns versus total water lost.

2.6 Project plan

Purpose and Objectives

The purpose of this project is to develop a simulation game called "Pipes in the Desert" where players control plumbers and saboteurs to transfer or disrupt water flow through a complex pipe system in a desert environment. The objectives include creating a functional and engaging game, implementing realistic mechanics for water transfer and sabotage, and providing an enjoyable experience for players.

Assumptions and Constraints

Budget: This software will be designed as a university project without any considered budget.
Staff: 6 members, all students in BSc. Computer Science Engineering program of Budapest University of Technology and Economics.
There is also a Quality Member: **Dr. Balla Katalin**
Schedule: This is started on 02.23.2026 and is going to end by 05.29.2026 with the workflow specified in the next section.
Skill Requirement: Java programming language in OOP with the usage of Java libraries such as Swing and JUnit. etc. Git/GitHub (Git Bash commands) will also be needed for version control and CI.

Workflow

Week 1 - Forming the teams
Week 2 - Requirements, project, functionally – documentation
Week 3-4 - Analysis Model - first version – documentation
Week 5 - Planning the Skeleton – documentation
Week 6 - Skeleton program – documentation and source code
Week 7-8 - Concept of Prototype – documentation
Week 9 - Detailed plans – documentation
Week 10 - Creating the Prototype part
Week 11 - Prototype program – documentation, source code
Week 12 - User interface specific – documentation
Week 13 - Creating the Complete part
Week 14 - Complete program – documentation and source code
Week 15 - Summary

Deliverables

Game Design Document	**To Be Delivered**
Software Prototype	**To Be Delivered**
Final Version Release	**To Be Delivered**

Supporting Processes

- Regular team meetings for progress updates and issue resolution.
- Continuous integration and version control for code management.

Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
2026.02.27 19:00	3 hours	Ilgın, Arda, Ibrahim, Hameez	→ Held initial team meeting to understand the project scope and requirements. → Defined schedule and task distribution. → Drafted initial Functional and Non-Functional Requirements. → Structured the document (sections, numbering, main headings). → Identified system actors, assumptions, and constraints. → Discussed subsystem division and overall system boundaries.
2026.02.28 14:00	3 hours	Yahya, Aasif	→ Reviewed team updates and existing document draft. → Yahya created the initial UML Use Case Diagram and defined actor relationships. → Aasif refined and reorganized Functional Requirements for clarity and consistency. → Added missing requirement details and improved formatting.
2026.02.01 12:00	6 hours	Ibrahim, Arda, Hameez, Ilgın, Yahya, Aasif	→ Conducted a full-team working session to finalize the document. → Reviewed and corrected Functional and Non-Functional Requirements. Completed and refined Use Case Diagram and detailed use case descriptions. → Ensured consistency between diagrams and written requirements. → Completed glossary and remaining documentation sections. → Performed final formatting, corrections, and preparation for submission.